TP — Réseaux — M1 Informatique

Mini-projet — Conception et programmation d'un protocole réseau

Jeu du pendu en réseau

Objectif	Concevoir puis implémenter un protocole applicatif client–serveur au-dessus de TCP.
Organisation	Travail en binôme.
Durée	12 h.
Livrables	Démonstration à la dernière séance ; dépôt sur Arche du code source et d'un compte-rendu.

Le projet ne consiste pas seulement à programmer un jeu : il faut concevoir un véritable protocole applicatif au-dessus de TCP.

Objectifs pédagogiques

- mettre en pratique la programmation socket TCP ;
- distinguer flux d'octets et messages applicatifs ;
- définir la syntaxe et la sémantique d'un protocole ;
- gérer l'état d'une session applicative ;
- implémenter un serveur concurrent ;
- gérer les erreurs, les déconnexions et la reprise d'une session ;
- faire le lien entre les couches Transport, Session, Présentation et Application ;
- en extension, explorer UDP, IPv6, TLS ou la conteneurisation.

Travail à rendre

1. **À la dernière séance, chaque binôme réalisera une démonstration du projet.**
2. **Sur Arche, déposer une archive contenant:**
    - le code source ;
    - un compte-rendu présentant l'architecture client–serveur ;
    - la spécification du protocole et les formats de messages ;
    - la machine à états ;
    - la gestion des erreurs ;
    - la gestion des sessions et des déconnexions ;
    - les principaux choix techniques.

Note : la présence aux séances de TP est obligatoire.

Partie 1 — Conception du protocole applicatif — 2 points

Avant de commencer l'implémentation, vous devez définir précisément le protocole permettant au client et au serveur de communiquer.

1.1 Service rendu

1. Quel service est rendu par le serveur ?
2. Quelles opérations peut demander le client ?
3. Quelles réponses le serveur peut-il retourner ?
4. Quelles informations sont conservées par le serveur pendant une partie ?
5. Qui fixe les règles du jeu : le client ou le serveur ?
6. Quelles vérifications doivent être réalisées côté serveur ?

Le serveur doit rester autoritaire sur l'état du jeu : le client ne décide pas lui-même qu'une partie est gagnée et ne modifie pas directement le mot à trouver.

1.2 Définition des messages

Définir les différents types de messages du protocole. Vous pouvez, par exemple, utiliser :

```
HELLO
NEW_GAME
LETTER
STATE
QUIT
ERROR
RESUME
```

Cette liste n'est pas imposée. Pour chaque message, préciser :

- son type et son rôle ;
- ses champs ;
- les champs obligatoires et facultatifs ;
- la réponse attendue ;
- les erreurs possibles.

Exemple conceptuel d'une requête :

```
{
 "type": "LETTER",
 "request_id": 17,
 "session_id": "S42",
 "payload": {
  "letter": "e"
 }
}
```

Exemple conceptuel de réponse :

```
{
 "type": "STATE",
 "request_id": 17,
 "session_id": "S42",
 "payload": {
  "word": "_e__e",
  "remaining": 4
 }
}
```

Le format est libre (JSON, texte structuré, binaire, etc.), mais il doit être documenté et cohérent.

Partie 2 — Flux TCP — 1 points

La communication entre le client et le serveur doit être réalisée avec des sockets TCP.

TCP transporte un flux d'octets, pas une suite de messages applicatifs.

Un appel à :

```
send(message1)
send(message2)
```

ne garantit pas que le récepteur effectuera :

```
recv(message1)
recv(message2)
```

Les données peuvent notamment être reçues comme plusieurs messages concaténés ou comme une partie seulement d'un message.

2.1 Framing

Choisir et implémenter un mécanisme permettant au récepteur de retrouver les limites des messages.

Solution possible A — Délimiteur

```
{"type":"LETTER","letter":"A"}\n
{"type":"STATE","word":"A__A_"}\n
```

Solution possible B — Longueur explicite

```
[ longueur sur 4 octets ][ message sur N octets ]
```

Dans le compte-rendu, expliquer :

- le mécanisme choisi ;
- comment un message complet est détecté ;
- comment plusieurs messages reçus dans un même recv() sont traités ;
- comment un message reçu partiellement est reconstitué.

Partie 3 — Session et machine à états — 1 points

Une partie doit être considérée comme une session applicative. Le serveur conserve l'état nécessaire à son déroulement.

```
session_id
joueur
mot à trouver
lettres proposées
nombre d'erreurs
état de la partie
```

3.1 Machine à états

Définir une machine à états représentant le cycle de vie d'une partie.

```
     NEW_GAME
       |
       v
    [ EN_COURS ]
   /   |   \
 WIN  LOSE  QUIT
  |    |    |
  v    v    v
GAGNÉ PERDU ABANDONNÉ
```

Le diagramme doit indiquer :

- les états ;
- les événements ;

- les transitions ;
- les messages autorisés dans chaque état.

Le serveur doit détecter les opérations incohérentes. Exemple : un message LETTER reçu alors que la partie est déjà terminée peut produire :

```
ERROR INVALID_STATE
```

Partie 4 — Gestion des erreurs — 1 points

Le protocole doit prévoir explicitement les erreurs applicatives.

```
INVALID_MESSAGE
INVALID_STATE
INVALID_LETTER
UNKNOWN_SESSION
PLAYER_NOT_ALLOWED
TIMEOUT
```

Chaque erreur doit contenir au minimum un code et une description. Exemple :

```
{
 "type": "ERROR",
 "code": "INVALID_LETTER",
 "message": "Une seule lettre est attendue."
}
```

Erreur de transport	Erreur applicative
connexion interrompue, timeout réseau, connexion impossible	lettre invalide, session inexistante, commande interdite dans l'état courant
Le transport ne fournit plus correctement le canal.	Le message arrive, mais l'opération demandée est invalide.

Une communication TCP peut fonctionner correctement alors que l'opération applicative échoue.

Partie 5 — Serveur concurrent — 2 points

Le serveur doit être capable de gérer plusieurs clients simultanément. Chaque client doit pouvoir jouer une partie indépendante.

```
Client A --\
Client B ----> Serveur
Client C ---->
Client D --/
```

L'implémentation est libre : threads, processus, programmation asynchrone ou boucle d'événements, selon le langage utilisé.

Le serveur doit clairement distinguer :

- la socket d'écoute ;
- les sockets associées aux différents clients.

La démonstration devra montrer plusieurs parties actives simultanément.

Partie 6 — Déconnexion et reprise d'une partie — 2 points

Une partie interrompue par une déconnexion doit pouvoir être reprise pendant une durée définie.

```
Client ---- session S42 ----> Serveur
        X coupure
```

```
Client ---- RESUME S42 ----> Serveur
                              |
                              v
                         état précédent
                              |
                              v
                         partie reprise
```

Le serveur peut conserver, par exemple :

```
session S42
joueur = Alice
mot = _E__E
essais restants = 4
lettres = A, E, R
état = EN_COURS
```

Préciser :

- comment est créé le session_id ;
- combien de temps une session déconnectée reste valide ;
- comment le client prouve qu'il reprend sa propre session ;
- ce qui se passe lorsque la session demandée n'existe plus.

6.1 Ambiguïté après une coupure

```
Client ---- LETTER E ----> Serveur
                             traite E
Client <--- STATE ---------X
                         réponse perdue
```

Le client ne sait pas nécessairement si la commande a été traitée ou si elle n'est jamais arrivée. Votre protocole doit expliquer comment cette situation est gérée.

Une possibilité consiste à utiliser un identifiant de requête :

```
request_id = 42

si la requête 42 a déjà été traitée :
  ne pas appliquer l'opération une deuxième fois
  renvoyer le résultat déjà associé à 42
```

Il n'est pas demandé d'implémenter une garantie « exactly-once » distribuée complète, mais le comportement en cas de retransmission doit être défini.

Partie 7 — Extensions — 6 points

Plusieurs extensions sont possibles, laissez libre cours à votre imagination pour proposer des extensions du jeu, par exemple

A — Jeu joueur contre joueur

Deux clients jouent l'un contre l'autre.

Gérer création de partie, attente d'un second joueur, association des joueurs, tours de jeu, fin de partie et déconnexion.

B — Salles de jeu

Permettre LIST_ROOMS, CREATE_ROOM, JOIN_ROOM et LEAVE_ROOM.

Plusieurs parties peuvent avoir lieu simultanément dans différentes salles.

C — Chat

Ajouter un message CHAT.

Préciser s'il s'agit d'un chat global, par salle ou limité à deux joueurs.

D — Chronomètre

Le serveur impose une durée maximale pour jouer.

Le temps doit être géré côté serveur.

F — Compatibilité IPv4 / IPv6

Faire fonctionner le serveur en IPv4 et IPv6.

Tester notamment 127.0.0.1 et ::1.

G — TLS

Protéger la communication client–serveur avec TLS.

Expliquer ce que TLS apporte et ce qu'il ne remplace pas dans le protocole applicatif.

Note : une interface graphique n'est pas obligatoire mais conseillée

Barème

Partie	Points
1. Conception du protocole applicatif	2
2. Framing du flux TCP	1
3. Session et machine à états	1
4. Gestion des erreurs	1
5. Serveur concurrent	2
6. Déconnexion et reprise	2
Extensions	7
Qualité générale du code et du rapport + évolution pendant les TPs	4
Total	**20**

Attendus pour la démonstration

1. démarrage du serveur ;
2. connexion d'un client ;
3. création d'une partie ;
4. plusieurs échanges client ↔ serveur ;
5. partie gagnée ou perdue ;
6. traitement d'un message invalide ;
7. plusieurs clients simultanés ;
8. déconnexion d'un client ;
9. reconnexion et reprise de sa session.